

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
Московский авиационный институт
(Национальный исследовательский университет)

Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806

Отчёт по лабораторной работе №2

Дисциплина: «Операционные системы и архитектура компьютера»

Тема: «Работа с процессами ОС»

Вариант: 18

Выполнил:
студент группы М8О-206БВ-25
Ширанков М.С.
Проверил:
Ассистент Центра ТОП-ИТ
Ефимов Д.А.

Москва 2026

Содержание

Цель работы **2**

Задание **2**

Ход работы **2**

Используемые системные вызовы **2**

Подготовка 3
Порождение дочерних процессов 4
Пересылка строк 4
Работа дочернего процесса 4
Завершение 5

Обработка ошибок **5**

Демонстрация работы **5**

Трассировка системных вызовов 6
Сводная статистика 7

Итог работы **8**

Приложение А. Исходный код программы **9**

A.1. master.c 9
A.2. slave.c 15
A.3. common.h 16
A.4. common.c 16
A.5. Makefile 17

Цель работы

Приобрести практические навыки управления процессами в ОС семейства UNIX: порождения дочерних процессов (`fork`), замены их образа другой программой (`exec`), перенаправления стандартных потоков (`dup2`) и обмена данными между процессами через неименованные каналы (`pipe`), а также корректной обработки ошибок всех используемых системных вызовов.

Задание

Родительский процесс создаёт два дочерних процесса. Первой строкой пользователь вводит в консоль родительского процесса имя файла, который будет открыт на запись для `child1`, второй строкой — аналогичное имя для `child2`. Родительский и дочерний процессы должны быть представлены разными программами.

Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в `pipe1` или `pipe2` в зависимости от правила фильтрации. Процессы `child1` и `child2` обрабатывают строки и пишут результаты своей работы в стандартный вывод. Все возможные ошибки должны корректно обрабатываться.

Правило фильтрации (вариант 18): нечётные строки отправляются в `pipe1`, чётные — в `pipe2`. Дочерние процессы удаляют из строк все гласные.

Ход работы

Решение состоит из двух программ и общего модуля (полный исходный код приведён в приложении А):

- `master.c` — родительский процесс: читает имена файлов, создаёт каналы, порождает дочерние процессы и распределяет строки между ними;
- `slave.c` — дочерняя программа: фильтр `stdin` → `stdout`, удаляющий гласные `aeiouAEIOU`;
- `common.c`, `common.h` — общие функции ввода-вывода (`read_retry`, `write_all`) и коды завершения.

Обе программы запускаются от одного и того же исполняемого файла `slave`: `child1` и `child2` — это два экземпляра одной программы, различающиеся только подключёнными к ним каналом и файлом. Сборка выполняется по `Makefile` из корня лабораторной работы:

```
make
./master
```

Ключевое решение: файл для дочернего процесса открывает родитель (`open_output_file`), а затем через `dup2` подставляет его дескриптор на место стандартного вывода ребёнка (`spawn_slave`). Поэтому дочерняя программа, как и требует задание, пишет результат в `stdout` и ничего не знает о файлах. Ошибка открытия файла при этом обнаруживается до порождения процессов.

Используемые системные вызовы

В программе используются следующие системные вызовы и функции стандартной библиотеки, являющиеся тонкими обёртками над ними:

Вызов	Назначение в программе	Где в коде
`read`	чтение имён файлов (по одному байту) и строк (блоками по 4 КБ) из `stdin`; чтение дочерним процессом своего канала	`read_retry`, `read_filename`, `forward_lines`, `main` (slave)
`write`	пересылка строк в каналы; запись результата дочерним процессом в `stdout`	`write_all`
`open`	открытие (создание, усечение) выходных файлов с флагом `O_CLOEXEC`	`open_output_file`
`fstat`	проверка того, что оба имени не указывают на один и тот же файл	`is_same_regular_file`
`pipe`	создание двух неименованных каналов `pipe1`, `pipe2`	`open_pipe`
`fcntl`	проверка открытости дескрипторов 0–2 (`F_GETFD`); установка флага `FD_CLOEXEC` на концы каналов	`check_std_fds`, `set_cloexec`
`fork`	порождение дочернего процесса	`spawn_slave`
`dup2`	перенаправление `stdin` ребёнка на канал, а `stdout` — на файл	`spawn_slave`
`execv`	замена образа дочернего процесса программой `slave`	`spawn_slave`
`sigaction`	игнорирование `SIGPIPE` в родителе: запись в канал без читателя возвращает `EPIPE`, а не убивает процесс	`start_children`
`close`	закрытие ненужных концов каналов; закрытие пишущих концов для передачи детям признака конца данных (EOF)	`close_fd`, `finish_children`
`waitpid`	ожидание завершения детей и разбор их кода возврата	`wait_child`
`_exit`	завершение ребёнка при ошибке `dup2`/`execv` без сброса унаследованных буферов `stdio`	`spawn_slave`

Подготовка

Подготовку выполняет функция `start_children`:

1. **Проверка стандартных дескрипторов.** Если программа запущена с закрытым `stdin` (`./master <&-`), то `pipe()` вернул бы свободный номер 0, и последующий `dup2` перепутал бы дескрипторы. Поэтому сначала `check_std_fds` с помощью `fcntl(fd, F_GETFD)` проверяет, что 0, 1 и 2 открыты.

2. **Чтение имён файлов** (`read_filename`). Имена читаются по одному байту до `\n`. Так родитель забирает из `stdin` ровно две строки, а остальные данные остаются в ядре для этапа пересылки. Длина имени ограничена `PATH_MAX`; завершающий `\r` отбрасывается.

3. **Открытие файлов** (`open_output_file`) с флагами `O_WRONLY`, `O_CREAT`, `O_TRUNC` и `O_CLOEXEC`. Затем `is_same_regular_file` сравнивает пары `(st_dev, st_ino)`:

два ребёнка с независимыми смещениями в одном обычном файле затирали бы данные друг друга.

4. **Создание каналов** (open_pipe) и установка FD_CLOEXEC на оба их конца (set_cloexec).

Порождение дочерних процессов

Каждый ребёнок запускается функцией spawn_slave. После fork ребёнок получает копию всех дескрипторов родителя. dup2 ставит нужный конец канала и файл на места 0 и 1, причём новый дескриптор флага FD_CLOEXEC не наследует. Все остальные копии ядро закрывает автоматически при execv:

```
child1 after fork:   0=stdin 1=stdout 2=stderr 3=file1 4=file2 5=pipe1[0] 6=pipe1[1]
    7=pipe2[0] 8=pipe2[1]
after dup2:          0=pipe1[0] 1=file1 2=stderr   3..8 are copies with CLOEXEC
after execv:         0=pipe1[0] 1=file1 2=stderr   (3..8 closed by the kernel)
```

Если бы child1 сохранил копию pipe2[1], child2 не получил бы конец данных, пока жив child1. Путь к программе slave строится от argv[0] (build_slave_path), поэтому master можно запускать из любого каталога. После каждого fork родитель закрывает свои копии читающего конца канала и файла (close_fd в start_children).

Обработчик SIGPIPE устанавливается после fork, так как диспозиция SIG_IGN наследуется через exec, а детям нужно поведение по умолчанию.

Пересылка строк

Пересылку выполняет функция forward_lines. Родитель читает stdin блоками по 4096 байт (константа kBlockSize, чтение — read_retry) и ищет в блоке символы \n функцией memchr. Фрагмент до перевода строки включительно уходит в текущий канал (write_all), после него получатель меняется: индекс получателя — номер строки по модулю kChildrenCount. Номер строки увеличивается только при встрече \n, поэтому строка, разрезанная границей блоков, целиком попадает в один канал:

```
block:  "abc\nde"  |  "f\nghi\n"
         pipe1 p2      p2  pipe1
```

Так выполняется требование о строках «произвольной длины»: строка нигде не хранится целиком, и даже строка, не помещающаяся в оперативную память, обрабатывается с постоянным расходом памяти. Проверено на строке длиной 1 ГБ: пиковое потребление памяти родителя около 0,9 МБ.

Работа дочернего процесса

Функция main программы slave читает stdin блоками, удаляет гласные на месте (проверка символа — is_vowel), сдвигая остальные байты к началу буфера, и записывает результат в stdout функцией write_all, которая повторяет write при частичной записи. Обработка побайтовая, поэтому место разреза строки родителем значения не имеет. read возвращает 0, когда все пишущие концы канала закрыты.

Завершение

Функция finish_children закрывает pipe1[1] и pipe2[1], дети получают EOF и завершаются, после чего для каждого из них вызывается wait_child, которая ожидает процесс через waitpid и разбирает его статус. Порядок важен: если ждать детей до закрытия каналов, возникнет взаимная блокировка — ребёнок ждёт конца данных, а родитель ждёт ребёнка.

Все ресурсы (дескрипторы, идентификаторы процессов) хранятся в структуре child_t, где значение -1 означает «не выделено». Функция подготовки start_children при ошибке просто возвращает код, а finish_children всегда закрывает всё открытое и дожидается всех запущенных процессов; main лишь вызывает эти этапы по очереди. Это позволяет обойтись без goto и корректно освобождать ресурсы на любом этапе, например если второй fork не удался, а первый ребёнок уже работает.

Обработка ошибок

Каждый системный вызов проверяется; вызовы read, write, open, waitpid, прерванные сигналом (EINTR), повторяются (read_retry, write_all, open_output_file, wait_child). Код завершения родителя — битовая маска, поэтому по нему видно все произошедшие ошибки одновременно:

Бит	Константа	Значение
1	kErrInput	ошибка чтения stdin, пустое или слишком длинное имя, один и тот же файл для обоих детей
2	kErrSetup	ошибка open, pipe, fork, sigaction, закрыт стандартный дескриптор
4	kErrChild1Write	child1 перестал читать pipe1
8	kErrChild2Write	child2 перестал читать pipe2
16	kErrChild1Status	child1 завершился с ошибкой или по сигналу
32	kErrChild2Status	child2 завершился с ошибкой или по сигналу

Максимальное значение 63 не пересекается с кодами 126, 127 и $128 + N$, которыми командная оболочка обозначает ошибки запуска и завершение по сигналу. Дочерний процесс завершается с кодом 126 (kChildSetupFailed), если не удался dup2, и 127 (kChildExecFailed), если не удался execv; сама программа slave возвращает 1 (kSlaveReadFailed) при ошибке чтения и 2 (kSlaveWriteFailed) при ошибке записи. Родитель расшифровывает эти коды в сообщениях (wait_child).

Если ребёнок аварийно завершился, запись в его канал возвращает EPIPE: forward_lines сообщает об этом, закрывает канал и продолжает пересылать строки второму ребёнку.

Демонстрация работы

Программа разрабатывалась на macOS; трассировка выполнена в контейнере Debian (ядро Linux 6.12, gcc 14.2, strace 6.13). Входные данные:

```
$ cat input.txt
out1.txt
```

```
out2.txt
Hello World
Operating Systems
abcdefghij
UNIX pipes
$ ./master < input.txt; echo "exit=$?"
exit=0
$ cat out1.txt
Hll Wrld
bcdfghj
$ cat out2.txt
prtng Systms
NX pps
```

Трассировка системных вызовов

```
strace -f -o trace.log -e trace=read,write,openat,pipe2,fcntl,clone,execve,\
    dup3,close,wait4,rt_sigaction,exit_group ./master < input.txt
```

Фрагмент трассы (строки динамического загрузчика опущены, PID 532 — `master`, 533 и 534 — дочерние процессы):

```
532   fcntl(0, F_GETFD)                 = 0
532   fcntl(1, F_GETFD)                 = 0
532   fcntl(2, F_GETFD)                 = 0
532   read(0, "o", 1)                   = 1
532   read(0, "u", 1)                   = 1
...
532   read(0, "\n", 1)                  = 1
532   openat(AT_FDCWD, "out1.txt", O_WRONLY|O_CREAT|O_TRUNC|O_CLOEXEC, 0644) = 3
532   openat(AT_FDCWD, "out2.txt", O_WRONLY|O_CREAT|O_TRUNC|O_CLOEXEC, 0644) = 4
532   pipe2([5, 6], 0)                  = 0
532   fcntl(5, F_SETFD, FD_CLOEXEC)     = 0
532   fcntl(6, F_SETFD, FD_CLOEXEC)     = 0
532   pipe2([7, 8], 0)                  = 0
...
532   clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    ...) = 533
532   close(5)                          = 0
533   dup3(5, 0, 0)                     = 0
532   close(3)                          = 0
533   dup3(3, 1, 0)                     = 1
533   execve("./slave", ["slave"], 0xffffc81fe8c8 /* 7 vars */) = 0
532   clone(...)                        = 534
532   close(7)                          = 0
534   dup3(7, 0, 0)                     = 0
532   close(4)                          = 0
534   dup3(4, 1, 0)                     = 1
532   rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[], sa_flags=0}, NULL, 8) = 0
534   execve("./slave", ["slave"], 0xffffc81fe8c8 /* 7 vars */) = 0
532   read(0, "Hello World\nOperating Systems\nab"..., 4096) = 52
532   write(6, "Hello World\n", 12)     = 12
532   write(8, "Operating Systems\n", 18) = 18
532   write(6, "abcdefghij\n", 11)      = 11
532   write(8, "UNIX pipes\n", 11)      = 11
532   read(0, "", 4096)                 = 0
```

```
532   close(6)                          = 0
532   close(8)                          = 0
532   wait4(533,  <unfinished ...>
533   read(0, "Hello World\nabcdefghij\n", 4096) = 23
533   write(1, "Hll Wrld\nbcdfghj\n", 17) = 17
534   read(0, "Operating Systems\nUNIX pipes\n", 4096) = 29
534   write(1, "prtng Systms\nNX pps\n", 20) = 20
533   read(0, "", 4096)                 = 0
533   exit_group(0)                     = ?
532   <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 533
534   read(0, "", 4096)                 = 0
534   exit_group(0)                     = ?
532   wait4(534, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 534
532   exit_group(0)                     = ?
```

По трассе видны все этапы работы:

- имена файлов читаются по одному байту (`read_filename`), а строки — одним блоком (`read(0, ..., 4096) = 52`) в `forward_lines`;
- glibc реализует `pipe()` через `pipe2(fds, 0)`, `fork()` — через `clone` с флагом `SIGCHLD`, а `dup2()` — через `dup3`;
- в ребёнке (`spawn_slave`) `dup3(5, 0)` подставляет `pipe1[0]` на место `stdin`, `dup3(3, 1)` — файл на место `stdout`;
- блок из 52 байт разрезается на четыре строки, которые поочерёдно уходят в дескрипторы 6 (`pipe1`) и 8 (`pipe2`);
- каждый ребёнок читает обе свои строки одним вызовом `read` и пишет результат одним `write` в дескриптор 1, то есть в файл;
- после `close(6)` и `close(8)` дети получают `read(...) = 0` и завершаются, а родитель собирает их статусы через `wait4` (`finish_children`). Строки `<unfinished ...>` и `<... resumed>` показывают, что процессы выполняются параллельно.

Сводная статистика

```
strace -f -c ./master < input.txt
```

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 70.17    0.001508         754         2           wait4
  9.35    0.000201           9        21           mmap
  7.21    0.000155          12        12           mprotect
  3.44    0.000074           9         8           munmap
  2.14    0.000046           1        27           read
  2.14    0.000046           7         6           write
  ...
  0.00    0.000000           0         4           dup3
  0.00    0.000000           0        11           fcntl
  0.00    0.000000           0         2           pipe2
  0.00    0.000000           0         1           rt_sigaction
  0.00    0.000000           0         2           clone
```

```
  0.00    0.000000           0         3           execve
------ ----------- ----------- --------- --------- ----------------
100.00    0.002149          13       156         6 total
```

Основное время уходит на `wait4`: родитель блокируется в нём, пока дети обрабатывают данные. Три вызова `execve` — запуск `master` и двух экземпляров `slave`; `mmap`, `mprotect` и `munmap` выполняет динамический загрузчик при старте каждого из трёх процессов.

Итог работы

В ходе лабораторной работы была реализована программа из двух процессов-обработчиков и управляющего процесса, взаимодействующих через неименованные каналы. Получены практические навыки работы с системными вызовами `fork`, `execv`, `dup2`, `pipe`, `waitpid` и `sigaction`.

Основные выводы:

- каждый лишний открытый конец канала — потенциальная взаимоблокировка, поэтому их удобно закрывать автоматически флагом `FD_CLOEXEC`;
- потоковая обработка блоками фиксированного размера позволяет работать со строками любой длины при постоянном расходе памяти;
- игнорирование `SIGPIPE` превращает аварию дочернего процесса в обрабатываемую ошибку `EPIPE`;
- порядок завершения важен: сначала закрыть пишущие концы каналов, затем ожидать дочерние процессы.

ПРИЛОЖЕНИЕ А

Исходный код программы

Исходный код также доступен в репозитории на GitHub.

A.1. master.c

```c
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <unistd.h>

#include "common.h"

#ifndef PATH_MAX
#define PATH_MAX 4096
#endif

#define SLAVE_NAME "slave"

enum { kChildrenCount = 2 };

// Биты кода завершения, могут сочетаться, 0 означает успех
static const unsigned int kErrInput = 0x01U; // ошибка чтения stdin или имени файла
static const unsigned int kErrSetup = 0x02U; // ошибка open/pipe/fork/sigaction
static const unsigned int kErrChild1Write = 0x04U; // child1 перестал читать pipe1
static const unsigned int kErrChild2Write = 0x08U; // child2 перестал читать pipe2
static const unsigned int kErrChild1Status = 0x10U; // child1 завершился с ошибкой
static const unsigned int kErrChild2Status = 0x20U; // child2 завершился с ошибкой

typedef struct {
    const char* label;
    unsigned int write_err;
    unsigned int status_err;
    char filename[PATH_MAX];
    int file_fd;
    int pipe_fd[2];
    pid_t pid;
} child_t;

static int check_std_fds(void) {
    int all_open = 1;
    for (int fd = 0; fd <= 2; fd++) {
        if (fcntl(fd, F_GETFD) == -1) {
            fprintf(stderr, "Standard descriptor %d is closed.\n", fd);
            all_open = 0;
        }
    }
    if (!all_open) {
        return 1;
    }
```

```
50      return 0;
51  }
52
53  static void close_fd(int* fd) {
54      if (*fd != -1) {
55          close(*fd);
56          *fd = -1;
57      }
58  }
59
60  static int set_cloexec(int fd) {
61      int flags = fcntl(fd, F_GETFD);
62      if (flags == -1) {
63          return -1;
64      }
65      return fcntl(fd, F_SETFD, (int)((unsigned int)flags | (unsigned int)FD_CLOEXEC));
66  }
67
68  static int open_pipe(int fds[2]) {
69      if (pipe(fds) == -1) {
70          perror("pipe");
71          return -1;
72      }
73      if (set_cloexec(fds[0]) == -1 || set_cloexec(fds[1]) == -1) {
74          perror("fcntl");
75          return -1;
76      }
77      return 0;
78  }
79
80  static int build_slave_path(const char* argv0, char* path, size_t size) {
81      const char* dir = "./";
82      int dir_len = 2;
83      if (argv0 != NULL) {
84          const char* slash = strrchr(argv0, '/');
85          if (slash != NULL) {
86              dir = argv0;
87              dir_len = (int)(slash - argv0 + 1);
88          }
89      }
90
91      int len = snprintf(path, size, "%.*s%s", dir_len, dir, SLAVE_NAME);
92      if (len < 0 || (size_t)len >= size) {
93          fprintf(stderr, "Path to the slave program is too long.\n");
94          return -1;
95      }
96      return 0;
97  }
98
99  static int read_filename(const char* label, char* name, size_t size) {
100     if (isatty(STDIN_FILENO)) {
101         fprintf(stderr, "Enter file name for %s: ", label);
102     }
103
104     size_t len = 0;
105     for (;;) {
106         char symbol = '\0';
107         ssize_t bytes_read = read_retry(STDIN_FILENO, &symbol, 1);
```

```
108         if (bytes_read == -1) {
109             perror("Failed to read input");
110             return -1;
111         }
112         if (bytes_read == 0 || symbol == '\n') {
113             break;
114         }
115         if (len + 1 >= size) {
116             fprintf(stderr, "File name for %s is too long.\n", label);
117             return -1;
118         }
119         name[len++] = symbol;
120     }
121 
122     if (len > 0 && name[len - 1] == '\r') {
123         len--;
124     }
125     name[len] = '\0';
126 
127     if (len == 0) {
128         fprintf(stderr, "No file name for %s.\n", label);
129         return -1;
130     }
131     return 0;
132 }
133 
134 static int open_output_file(const char* filename) {
135     int fd = -1;
136     do {
137         fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC | O_CLOEXEC, 0644);
138     } while (fd == -1 && errno == EINTR);
139 
140     if (fd == -1) {
141         fprintf(stderr, "Cannot open '%s' for writing: %s\n", filename, strerror(errno)
                );
142     }
143     return fd;
144 }
145 
146 static int is_same_regular_file(int fd1, int fd2) {
147     struct stat first;
148     struct stat second;
149     if (fstat(fd1, &first) == -1 || fstat(fd2, &second) == -1) {
150         perror("fstat");
151         return -1;
152     }
153     return S_ISREG(first.st_mode) && first.st_dev == second.st_dev && first.st_ino ==
            second.st_ino;
154 }
155 
156 static pid_t spawn_slave(const char* slave_path, int in_fd, int out_fd) {
157     pid_t pid = fork();
158     if (pid != 0) {
159         return pid;
160     }
161 
162     if (dup2(in_fd, STDIN_FILENO) == -1 || dup2(out_fd, STDOUT_FILENO) == -1) {
163         perror("dup2");
```

```
        _exit(kChildSetupFailed);
    }

    char* const kArgs[] = {(char*)SLAVE_NAME, NULL};
    execv(slave_path, kArgs);

    fprintf(stderr, "Cannot execute '%s': %s\n", slave_path, strerror(errno));
    _exit(kChildExecFailed);
}

static int wait_child(const child_t* child) {
    int status = 0;
    pid_t waited = 0;
    do {
        waited = waitpid(child->pid, &status, 0);
    } while (waited == -1 && errno == EINTR);

    if (waited == -1) {
        fprintf(stderr, "waitpid for %s failed: %s\n", child->label, strerror(errno));
        return 1;
    }

    if (WIFSIGNALED(status)) {
        fprintf(stderr, "%s was killed by signal %d.\n", child->label, WTERMSIG(status)
            );
        return 1;
    }

    int code = WEXITSTATUS(status);
    switch (code) {
        case 0:
            return 0;
        case kChildExecFailed:
            fprintf(stderr, "%s could not execute the slave program.\n", child->label);
            break;
        case kChildSetupFailed:
            fprintf(stderr, "%s failed to set up its descriptors.\n", child->label);
            break;
        case kSlaveReadFailed:
            fprintf(stderr, "%s failed to read its pipe.\n", child->label);
            break;
        case kSlaveWriteFailed:
            fprintf(stderr, "%s failed to write its file.\n", child->label);
            break;
        default:
            fprintf(stderr, "%s finished with exit code %d.\n", child->label, code);
    }
    return 1;
}

static unsigned int start_children(child_t children[kChildrenCount], const char* argv0
    ) {
    char slave_path[PATH_MAX];
    if (build_slave_path(argv0, slave_path, sizeof(slave_path)) != 0) {
        return kErrSetup;
    }

    for (int i = 0; i < kChildrenCount; i++) {
```

```
220         if (read_filename(children[i].label, children[i].filename, sizeof(children[i].
                filename)) != 0) {
221             return kErrInput;
222         }
223     }
224 
225     for (int i = 0; i < kChildrenCount; i++) {
226         children[i].file_fd = open_output_file(children[i].filename);
227         if (children[i].file_fd == -1) {
228             return kErrSetup;
229         }
230     }
231 
232     int same = is_same_regular_file(children[0].file_fd, children[1].file_fd);
233     if (same == 1) {
234         fprintf(stderr, "Both children cannot write to the same file.\n");
235         return kErrInput;
236     }
237     if (same == -1) {
238         return kErrSetup;
239     }
240 
241     for (int i = 0; i < kChildrenCount; i++) {
242         if (open_pipe(children[i].pipe_fd) != 0) {
243             return kErrSetup;
244         }
245     }
246 
247     for (int i = 0; i < kChildrenCount; i++) {
248         children[i].pid = spawn_slave(slave_path, children[i].pipe_fd[0], children[i].
                file_fd);
249         if (children[i].pid == -1) {
250             fprintf(stderr, "fork for %s failed: %s\n", children[i].label, strerror(
                    errno));
251             return kErrSetup;
252         }
253         close_fd(&children[i].pipe_fd[0]);
254         close_fd(&children[i].file_fd);
255     }
256 
257     struct sigaction action = {0};
258     action.sa_handler = SIG_IGN;
259     sigemptyset(&action.sa_mask);
260     if (sigaction(SIGPIPE, &action, NULL) == -1) {
261         perror("sigaction");
262         return kErrSetup;
263     }
264     return 0;
265 }
266 
267 static unsigned int forward_lines(child_t children[kChildrenCount]) {
268     if (isatty(STDIN_FILENO)) {
269         fprintf(stderr, "Enter lines (Ctrl+D to finish):\n");
270     }
271 
272     char buf[kBlockSize];
273     size_t line_index = 0;
274     int alive = kChildrenCount;
```

```
    unsigned int flags = 0;

    while (alive > 0) {
        ssize_t bytes_read = read_retry(STDIN_FILENO, buf, sizeof(buf));
        if (bytes_read == 0) {
            break;
        }
        if (bytes_read == -1) {
            perror("Failed to read input");
            flags |= kErrInput;
            break;
        }

        char* start = buf;
        char* end = buf + bytes_read;
        while (start < end) {
            char* newline = memchr(start, '\n', (size_t)(end - start));
            char* piece_end = end;
            child_t* dst = &children[line_index % kChildrenCount];
            if (newline != NULL) {
                piece_end = newline + 1;
                line_index++;
            }

            int* fd = &dst->pipe_fd[1];
            if (*fd != -1 && write_all(*fd, start, (size_t)(piece_end - start))) {
                if (errno == EPIPE) {
                    fprintf(stderr, "%s stopped reading; its next lines are dropped.\n",
                        dst->label);
                } else {
                    fprintf(stderr, "Write to %s failed: %s\n", dst->label, strerror(
                        errno));
                }
                close_fd(fd);
                alive--;
                flags |= dst->write_err;
            }
            start = piece_end;
        }
    }
    return flags;
}

static unsigned int finish_children(child_t children[kChildrenCount]) {
    for (int i = 0; i < kChildrenCount; i++) {
        close_fd(&children[i].pipe_fd[0]);
        close_fd(&children[i].pipe_fd[1]);
        close_fd(&children[i].file_fd);
    }

    unsigned int flags = 0;
    for (int i = 0; i < kChildrenCount; i++) {
        if (children[i].pid > 0 && wait_child(&children[i]) != 0) {
            flags |= children[i].status_err;
        }
    }
    return flags;
}
```

```c

int main(int argc, char* argv[]) {
    if (check_std_fds() != 0) {
        return (int)kErrSetup;
    }

    child_t children[kChildrenCount] = {
        {
            .label = "child1",
            .write_err = kErrChild1Write,
            .status_err = kErrChild1Status,
            .file_fd = -1,
            .pipe_fd = {-1, -1},
            .pid = -1,
        },
        {
            .label = "child2",
            .write_err = kErrChild2Write,
            .status_err = kErrChild2Status,
            .file_fd = -1,
            .pipe_fd = {-1, -1},
            .pid = -1,
        },
    };

    const char* argv0 = NULL;
    if (argc > 0) {
        argv0 = argv[0];
    }
    unsigned int result = start_children(children, argv0);
    if (result == 0) {
        result = forward_lines(children);
    }
    return (int)(result | finish_children(children));
}
```

A.2. slave.c

```c
#include <errno.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#include "common.h"

static int is_vowel(char symbol) {
    return symbol != '\0' && strchr("aeiouAEIOU", symbol) != NULL;
}

int main(void) {
    char buf[kBlockSize];
    ssize_t bytes_read = 0;

    while ((bytes_read = read_retry(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        size_t len = 0;
        for (ssize_t i = 0; i < bytes_read; i++) {
            if (!is_vowel(buf[i])) {
                buf[len++] = buf[i];
```

```
            }
        }

        if (write_all(STDOUT_FILENO, buf, len) != 0) {
            fprintf(stderr, "slave[%d]: write failed: %s\n", (int)getpid(), strerror(
                errno));
            return kSlaveWriteFailed;
        }
    }

    if (bytes_read == -1) {
        fprintf(stderr, "slave[%d]: read failed: %s\n", (int)getpid(), strerror(errno))
            ;
        return kSlaveReadFailed;
    }
    return 0;
}
```

A.3. common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <stddef.h>
#include <sys/types.h>

enum {
    kBlockSize = 4096,
};

// Коды завершения дочернего процесса, не дошедшего до main() программы slave.
enum {
    kChildSetupFailed = 126,
    kChildExecFailed = 127,
};

// Коды завершения программы slave.
enum {
    kSlaveReadFailed = 1,
    kSlaveWriteFailed = 2,
};

ssize_t read_retry(int fd, void* buf, size_t len);

int write_all(int fd, const void* buf, size_t len);

#endif
```

A.4. common.c

```
#include "common.h"

#include <errno.h>
#include <stddef.h>
#include <unistd.h>

ssize_t read_retry(int fd, void* buf, size_t len) {
```

```
    ssize_t bytes_read = 0;
    do {
        bytes_read = read(fd, buf, len);
    } while (bytes_read == -1 && errno == EINTR);
    return bytes_read;
}

int write_all(int fd, const void* buf, size_t len) {
    const char* cursor = buf;
    while (len > 0) {
        ssize_t written = write(fd, cursor, len);
        if (written == -1) {
            if (errno == EINTR) {
                continue;
            }
            return -1;
        }
        cursor += written;
        len -= (size_t)written;
    }
    return 0;
}
```

A.5. Makefile

```
CC ?= cc
CFLAGS ?= -std=c11 -Wall -Wextra -Wpedantic -O2
CPPFLAGS += -D_POSIX_C_SOURCE=200809L -Isrc

vpath %.c src
vpath %.h src

all: master slave

master: master.o common.o
  $(CC) $(LDFLAGS) -o $@ $^

slave: slave.o common.o
  $(CC) $(LDFLAGS) -o $@ $^

master.o slave.o common.o: common.h

clean:
  rm -f master slave *.o

.PHONY: all clean
```